Battle-testing OmniPaxos with Jepsen

1. Introduction

Consensus protocols such as Paxos and Raft provide strong theoretical guarantees including agreement, leader completeness, and safety under crash failures. However, formal correctness proofs apply to the algorithmic model and do not automatically guarantee implementation correctness. Practical systems may violate safety due to concurrency bugs, improper read handling, network edge cases, or incorrect client semantics.

This project evaluates the following hypothesis:

> The OmniPaxos key-value store implementation preserves linearizability under aggressive network partitioning and node failures.

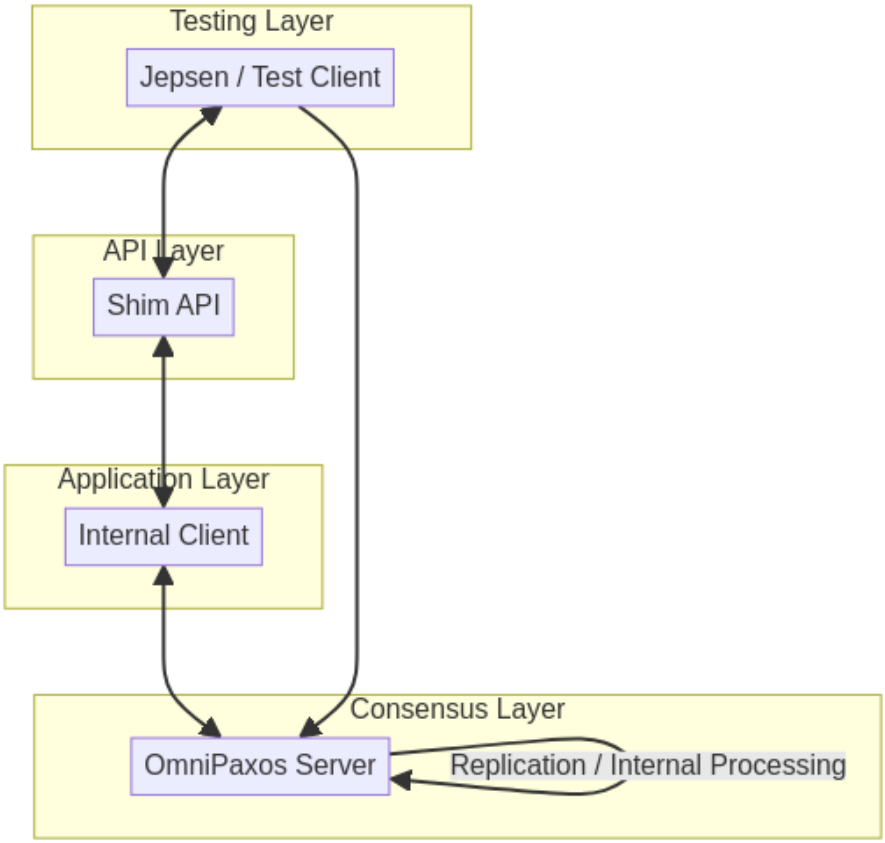
To test this hypothesis, we extended the OmniPaxos KV example with a programmable HTTP shim and subjected it to randomized fault injection using a Jepsen test suite. The system was tested under concurrent workloads, network partitions, and node crashes. Operation histories were then analyzed by the Knossos linearizability checker. We additionally implemented persistent storage to verify that the system recovers correctly after crashes without losing committed data.

2. System Architecture

2.1 Layered Design

The modified system consists of four logical layers:

1. **Testing Layer** – Jepsen orchestrates the test: it drives the client, controls the nemesis, and collects the operation history.
2. **API Layer** – An HTTP shim (Axum) exposes `PUT` and `GET` endpoints on each node, translating HTTP requests into internal commands.
3. **Application Layer** – The internal client manages the connection to a single OmniPaxos server, queues pending requests, and handles reconnection.
4. **Consensus Layer** – OmniPaxos servers replicate all operations through a distributed log before applying them to the key-value store.



All externally visible operations are routed through the consensus layer before completion. This ensures a single globally ordered log of operations.

2.2 Deployment

The cluster runs as six Docker containers: three server nodes (`s1`, `s2`, `s3`) and three client/shim nodes (`c1`, `c2`, `c3`). Each client is pinned to its corresponding server, with `c1` connecting to `s1`, and so on. The HTTP shims listen on port 3000 inside their containers, exposed on the host as `localhost:3001`, `3002`, and `3003`. Jepsen reaches each container via SSH over loopback addresses (`127.0.0.2`–`127.0.0.4`) and issues HTTP requests to the corresponding shim endpoint — for example, a request for node `127.0.0.2` goes to `http://localhost:3001`. This means Jepsen's SSH connections are unaffected by the Docker-level network partitions applied by the partition nemesis.

3. HTTP Shim and Client Integration

3.1 Motivation

The original OmniPaxos example was not designed for automated black-box testing. It relied on manual interaction and internal networking. To enable Jepsen-style testing, we implemented an HTTP shim that exposes a deterministic API.

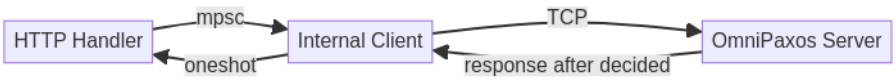
3.2 API Design

The shim exposes two endpoints:

- `POST /put`: accepts a JSON body `{ "key": "...", "value": "..." }` and performs a write.
- `GET /get/:key`: returns the current value for the given key.

CAS (compare-and-swap) was not implemented. The single-key workload with concurrent reads and writes is sufficient to detect the consistency violations we are interested in, and CAS would require additional server-side logic to be atomic through consensus.

Each HTTP request is converted into an `ApiCommand` and sent over an async channel (`mpsc`) to the internal client. A `oneshot` response channel is attached so the HTTP handler can await the result. This ensures that the HTTP response corresponds exactly to the decided log entry; the caller never receives a response before the operation is committed.



3.3 Timeout and Indeterminate States

There are two independent timeouts in the system. The shim applies a **10-second internal timeout** to each request waiting on consensus: if no decision arrives within that window, the HTTP handler returns an error string. Separately, the Jepsen client uses a **3-second socket timeout**, which fires first during node kills and allows the test to quickly record in-flight writes as `:info` without waiting for the full shim timeout. Jepsen records these operations with type `:info` (indeterminate), meaning the operation may or may not have succeeded. Knossos handles `:info` operations conservatively, considering all possible orderings when checking linearizability.

The socket timeout also controls how quickly `:info` operations are resolved during fault windows. A shorter timeout means fewer operations are simultaneously indeterminate, which directly reduces the search space Knossos must explore.

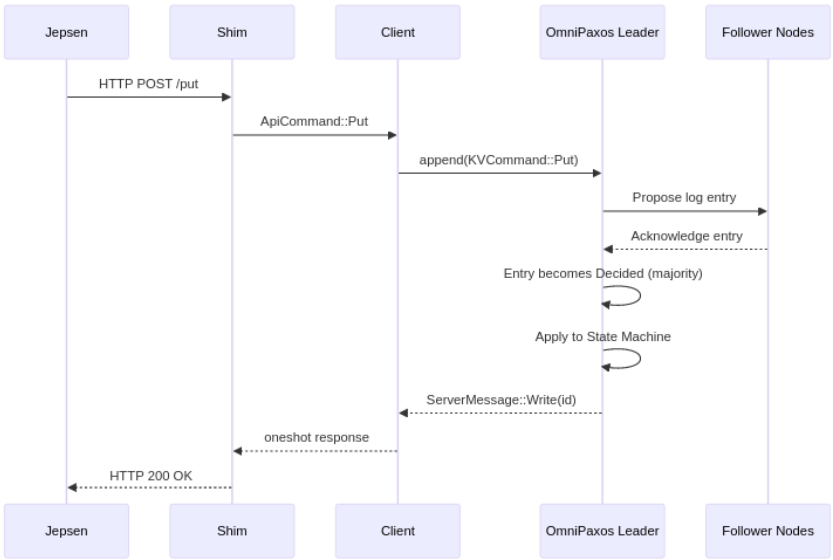
3.4 Reconnection

When the client detects a dropped connection (via a 2-second polling interval), it fails all pending requests immediately and enters a reconnection loop with 2-second retries. This prevents Jepsen from hanging indefinitely and allows the test to continue issuing operations while the cluster recovers.

4. Operation Flow and Linearizability

4.1 Write Path

A write operation follows this sequence:

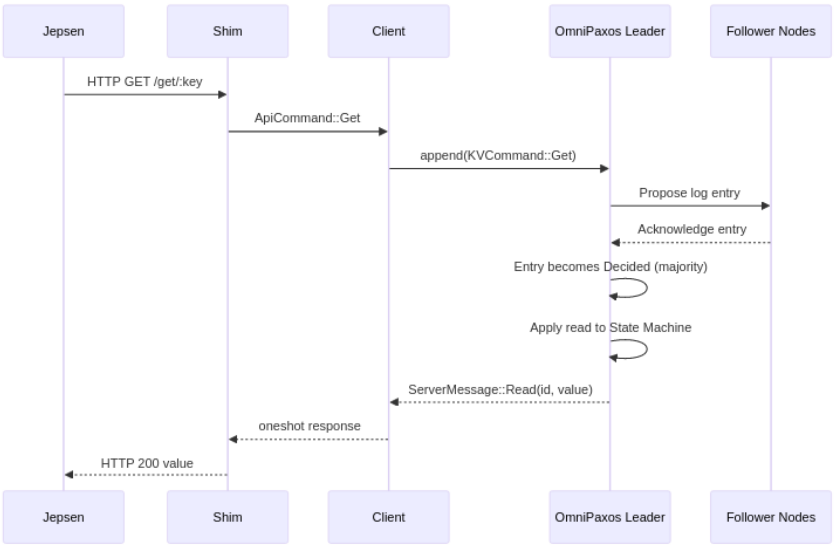


The linearization point occurs when the log entry becomes **decided**, meaning a majority of nodes have acknowledged it. The client receives a response only after this point.

4.2 Read Path and Linearizable Reads

A naive implementation might serve reads directly from the leader's local state. This is tempting because it avoids a consensus round, but it is **not linearizable**. If a leader is partitioned from the rest of the cluster, it may not know that a new leader has been elected. It would then serve stale values to clients, violating linearizability, which requires every read to reflect all writes that completed before it.

Our solution is to route reads through consensus just like writes. `KVCommand::Get` is appended to the OmniPaxos log, and the response is only sent after the entry is decided:



This approach has a clear linearization guarantee: a `Get` issued at time `t` is ordered in the log relative to all concurrent `Put` operations. Any write that returned before `t` must have been decided at a lower log index, and therefore its effect is visible to the read. A minority leader cannot serve reads at all; it cannot achieve quorum and will not decide any entries.

The trade-off is that reads are slower (a full consensus round), but correctness is guaranteed.

5. Fault Injection

We implemented three Jepsen test scenarios, each targeting a different failure mode. All tests use a 3-node cluster and run Knossos linearizability verification on the collected history.

5.1 Nemesis Kill (`nemesis_kill`)

This test kills a random server node with `SIGKILL` and restarts it after 30 seconds. At all times, 2 out of 3 nodes remain alive, preserving quorum. Operations are generated at **2 ops/sec per worker** (0.5 s average stagger) across 3 concurrent workers.

Generator pattern:

```
sleep 5s → kill random node → sleep 30s (re-election) → restart → sleep 20s → repeat
```

A key design decision is what happens to requests in flight when a node is killed. If Jepsen was connected to the killed node's shim, the TCP connection drops and all pending requests are failed immediately with an error. Jepsen records these as `:info`. If the killed node was the leader, OmniPaxos triggers a re-election;

surviving nodes detect the missing heartbeat and elect a new leader within a few election timeouts (configured at 500 ms).

5.2 Nemesis Partition (`nemesis_partition`)

This test injects network partitions using `iptables DROP` rules on Docker's internal network. It induces a split-brain scenario by dividing the 3-node cluster into two halves, typically isolating one node from the other two. Operations are generated at **2 ops/sec per worker** across 3 concurrent workers.

Generator pattern:

```
sleep 10s → split-brain partition → sleep 20s (minority loses quorum) → heal → sleep 10s → r
```

The partition is applied at the Docker internal IP layer (not the SSH loopback addresses), which means it affects OmniPaxos inter-node communication but not Jepsen's SSH connections. The minority partition (1 node) loses quorum and cannot commit new entries. If the leader is in the minority, it cannot make progress and any writes directed to it will timeout. When the partition heals, OmniPaxos performs ballot reconciliation to bring the isolated node back into the consistent state.

5.3 Nemesis Recovery (`nemesis_recovery`)

This test kills a single random node and restarts it after 30 seconds, using a longer post-restart stabilization window (30 s vs 20 s in `nemesis_kill`) to allow persistent state to load before the next cycle. At all times, 2 out of 3 nodes remain alive, so quorum is preserved throughout. The test is specifically designed to exercise persistent storage: the restarted node must reconstruct its state from disk before rejoining the cluster. Operations are generated at **2 ops/sec per worker** across 3 concurrent workers over a 300-second run. The recovery mechanism is described in detail in Section 6.

6. Persistent Storage and Recovery

6.1 Motivation

Without persistence, a crashed node restarts with empty state and must receive the full log from surviving peers before it can rejoin. Persistent storage avoids this by letting a node reconstruct most of its state locally, and provides durability guarantees so that committed data survives crashes.

6.2 Two-Layer Persistence

We implement persistence at two separate layers because they serve fundamentally different purposes and operate at different levels of the stack.

OmniPaxos log (RocksDB). OmniPaxos's `PersistentStorage` backend stores the consensus protocol state — ballot number, proposed log entries, and the

decided index — in a RocksDB database at `/app/logs/omnipaxos-node-{id}`. This layer is owned and managed entirely by OmniPaxos. Its purpose is to allow the consensus engine to resume correctly after a crash: without it, a restarted node would appear to OmniPaxos as a brand-new participant with an empty ballot and no log, forcing it to receive and re-replicate the entire history from surviving peers. With RocksDB, OmniPaxos can restore its ballot state and log independently of the rest of the cluster.

Database snapshot (JSON). The key-value store state is periodically snapshotted to `/app/logs/server-{id}-snapshot.json`. Each snapshot records the full database contents and the `decided_idx` at the time of the snapshot. Writes use an atomic `rename` pattern (write to `.tmp`, then rename) with `fsync` to prevent partial writes from corrupting the snapshot file. With `SNAPSHOT_INTERVAL = 1`, a snapshot is written after every decided entry. This layer is owned and managed by the application, not by OmniPaxos.

The key difference is that **RocksDB stores the log of operations** (the what-happened record used by consensus), while **JSON snapshots store the derived state** (the result of applying those operations to the database). A node that only had RocksDB would need to replay every log entry from the beginning on each restart to reconstruct the KV state — a process that grows linearly with the log length. The snapshot bounds that replay: on restart, the node loads the KV state directly from the snapshot and only replays entries that were decided after the snapshot was taken. With `SNAPSHOT_INTERVAL = 1`, that is at most one entry.

Neither layer alone is sufficient: without RocksDB, OmniPaxos cannot resume consensus correctly; without the JSON snapshot, recovery becomes increasingly expensive as the log grows.

6.3 Recovery Sequence

When a server restarts:

1. It loads the latest database snapshot from disk, restoring the KV state and the `decided_idx` at the time the snapshot was taken.
2. It opens the RocksDB storage, which independently restores OmniPaxos's consensus state: ballot number, log entries, and decided index.
3. As new entries are decided (either replayed from the local RocksDB log or received from peers), the server skips any entry whose log index is $\leq$ `snapshot_decided_idx` — these are already reflected in the restored KV state. Only entries above that index are applied to the database.
4. The server reconnects to peers and participates in leader election normally.

This avoids both re-applying entries that are already captured in the snapshot and missing entries that were decided after the last snapshot was written.

6.4 Leader Recovery

If the killed node was the leader, a new leader is elected while it is down. When it restarts, it discovers a higher ballot via peer messages and steps down. It then catches up on any entries it missed while offline. Because OmniPaxos uses ballot numbers to prevent old leaders from committing, there is no risk of split-brain.

7. Experimental Results

All three test suites were run against the 3-node Docker cluster. Each test collects an operation history (a timestamped log of invocations and responses) and passes it to Knossos for linearizability checking.

7.1 Results Summary

Test	Nemesis	Duration	Total ops	Knossos
kill	Random node SIGKILL	60 s	125	valid
partition	Split-brain (iptables)	60 s	115	valid
recovery	Kill + persistent restart	300 s	564	valid

Operation counts (ok = confirmed, info = indeterminate write, fail = refused read):

Test	ok writes	ok reads	info writes	fail reads
kill	64	49	5	0
partition	55	44	11	0
recovery	234	278	22	2

All histories were verified as linearizable by Knossos. For example, the recovery test ended with:

```
{:linear {:valid? true, :model #knossos.model.Register{:value 62}}, :valid? true}
```

No lost writes or stale reads were detected in any run.

7.2 Observations

During partitions, 11 writes became `:info` as operations directed at the minority node timed out (3-second socket timeout). Zero reads failed with `:ok` from a stale state — the minority node could not achieve quorum and therefore could not decide any entries, so no stale read could be returned. Once the partition healed, the system resumed within a few election cycles.

During kills, 5 writes became `:info` in the kill test and 22 in the longer recovery test. Clients connected to the killed node's shim received connection errors as

soon as the TCP connection dropped. No reads were served during the dead window (0 `:fail` reads in the kill test), confirming that the surviving nodes continued operating on the majority side and the killed node served no operations while down.

After recovery, the restarted node rejoined the cluster and caught up on missed entries. The 300-second recovery test produced 2 `:fail` reads, both during the brief window when all three shims were simultaneously unreachable due to reconnection timing. Knossos verified the full 564-operation history as linearizable.

8. Discussion

8.1 Linearizable Reads Trade-off

Routing reads through consensus is the safest approach, but it doubles the latency of read operations compared to leader-local reads. In read-heavy workloads this can be a meaningful overhead, and it is a trade-off we consciously accept in exchange for linearizability.

8.2 Indeterminate Operations

A recurring challenge in distributed testing is handling operations that may or may not have succeeded. Our shim returns errors on timeout or disconnection, which Jepsen records as `:info`. If these were recorded as `:ok`, Knossos could find false violations (a write appears in the history but the system never acknowledged it). If they were `:fail`, we might miss real violations (a write was committed but not acknowledged). The `:info` type correctly expresses the ambiguity, and Knossos handles it by exploring all possible orderings.

8.3 Single-Key Workload

All three tests operate on a single shared key (`jepsen-key`). This is intentional: using a single register maximizes contention and makes linearizability violations easiest to detect. Knossos uses the **register** model, which expects at most one value at any time and can detect stale reads and lost writes efficiently on a single key.

---

9. Conclusion

We extended the OmniPaxos key-value store with an HTTP shim suitable for automated black-box testing, implemented a Jepsen test suite covering node crashes, network partitions, and crash recovery, and verified linearizability of the operation histories with Knossos.

The key design decision was to route all read operations through the consensus log, rather than serving them locally from the leader. This guarantees that reads

are always consistent with the most recently committed writes, even under leader changes and partitions. The cost is an extra consensus round per read, which we accept in exchange for linearizability.

The persistent storage implementation ensures that a crashed node can recover and rejoin the cluster without data loss. The two-layer approach, using RocksDB for the OmniPaxos log and atomic JSON snapshots for the KV state, correctly handles the interplay between the consensus layer and the application state machine.

All experiments confirmed the initial hypothesis: the OmniPaxos implementation preserves linearizability under aggressive fault injection.